

Models of Higher Brain Function

Lecture Notes

Arjun Puri

Summer 2026

Contents

Preface	2
Notation	3
1 Course Overview and Modeling Strategy	4
1.1 What makes a model useful?	4
1.2 A common abstraction stack	5
1.3 Reusable model code	5
1.4 Next steps	5
2 Lecture 1: Visual Perception and Deep Learning	6
2.1 Why vision belongs in a course on higher brain function	6
2.1.1 Why this problem is harder than it feels	7
2.2 Invariant object recognition	7
2.2.1 Why pixels are a bad format for recognition	7
2.2.2 What makes a representation useful	7
2.3 The ImageNet turning point	8
2.3.1 Why AlexNet changed the conversation	8
2.3.2 From AlexNet to ResNet	8
2.4 What deep networks compute	9
2.4.1 Why nonlinearity matters	9
2.4.2 Why convolution is useful for images	9
2.4.3 Backpropagation and the credit-assignment problem	9
2.5 Deep networks as models of ventral visual cortex	10
2.5.1 Why compare representational geometry	10
2.5.2 The RSA construction	10
2.5.3 Comparing a brain RDM with a model RDM	11
2.5.4 How to interpret an RSA result	12
2.5.5 What the comparisons show	12
2.6 Deep networks as experimental tools	13
2.7 What deep networks still miss	13
2.7.1 Adversarial fragility	13
2.7.2 Biological caveats	13

2.8	Summary of concepts introduced in Lecture 1	14
2.9	Conceptual takeaways from Lecture 1	14
3	Lecture 2: Models of Attention	15
3.1	The behavioral anchor: pop-out and serial search	15
3.2	The spotlight as a minimal gain model	16
3.3	Saliency maps	16
3.3.1	Center-surround contrast	17
3.3.2	Normalization	17
3.3.3	From feature maps to a master map	17
3.3.4	Winner-take-all and inhibition of return	19
3.3.5	What saliency maps explain	19
3.4	Attention on the neural level	19
3.5	Stimulus interactions and suppression	19
3.6	Biased competition	20
3.6.1	Sum and difference variables	20
3.6.2	Divisive normalization as a related form	22
3.7	Cueing, delay activity, and target trials	22
3.8	Relation to transformer self-attention	22
3.9	Frontier directions	23
3.10	Summary of concepts introduced in Lecture 2	23
3.11	Conceptual takeaways from Lecture 2	23
4	Mathematical Synthesis: Higher Brain Function as Inference	24
4.1	Latent States	24
4.2	Prediction And Error	25
4.3	Representations As Coordinates	25
4.4	Decisions Couple Belief To Action	26

Preface

These notes study higher brain function as a set of modeling problems: perception, attention, memory, decision making, and control are treated as latent computations inferred from behavior and neural data. The course therefore moves between three levels of explanation: what problem the organism appears to solve, what algorithm could solve it, and what neural mechanism could plausibly implement that algorithm.

The notes are intentionally mathematical but not notation-driven. Equations are included when they clarify a representational claim, a dynamical mechanism, or an inference procedure. A useful model should compress data, expose its assumptions, and make clear predictions about where it should fail.

Berlin, April 2026

Notation

Dynamical systems

Symbol	Meaning
x_t	State at time t
u_t	Input or control signal
$f(\cdot)$	State transition or update map
τ	Characteristic time constant

Representations and learning

Symbol	Meaning
h	Hidden representation
W	Weight matrix
L	Objective or loss function
∇L	Gradient of the objective

Probability and inference

Symbol	Meaning
$p(z \mid x)$	Posterior distribution
$\mathbb{E}[\cdot]$	Expectation
$\text{Var}[\cdot]$	Variance
$\arg \max$	Optimization operator

1 Course Overview and Modeling Strategy

Higher brain function is a modeling problem before it is a tooling problem. Memory, planning, inference, abstraction, and decision making are all latent processes: we rarely observe them directly, but instead infer them from behavior, neural activity, and task structure. The course therefore asks how to build models whose hidden variables are meaningful, constrained, and experimentally testable.

The working standard is that every model should answer four questions. What is the observable behavior? Which latent variables explain it? What update rule moves those variables through time? What would count as evidence against the model? Keeping these questions explicit prevents the notes from becoming a catalog of architectures without interpretation.

1.1 What makes a model useful?

A model of higher brain function should do more than fit observations. It should compress behavior into a tractable set of assumptions, explain why certain error patterns appear, and suggest new experiments or interventions. In cognitive and systems neuroscience, this usually means balancing three goals:

- computational clarity,
- biological plausibility,
- empirical testability.

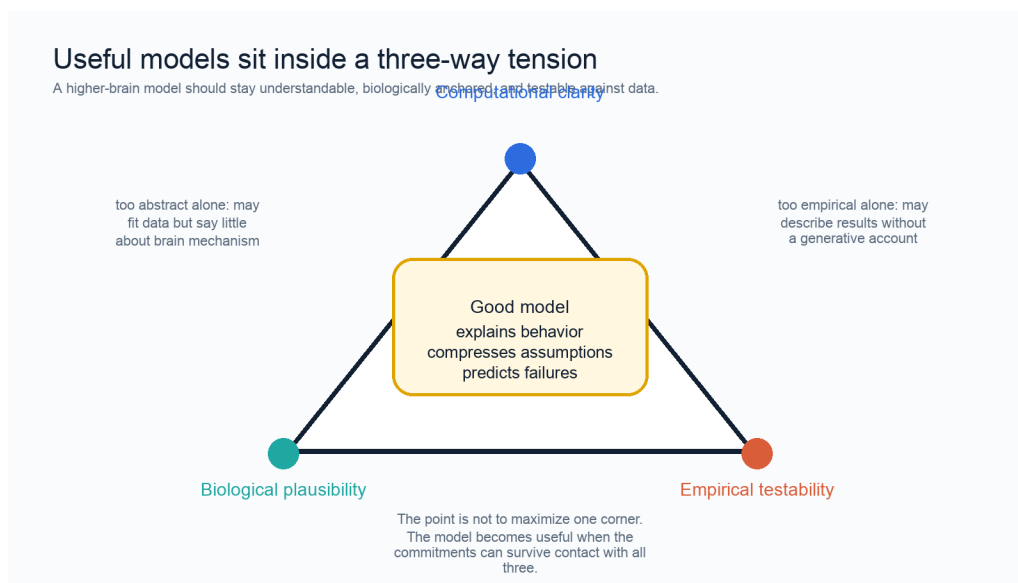


Figure 1: Why useful models live in tension. Strong higher-brain models stay interpretable, stay anchored to plausible mechanisms, and still make contact with measurable data.

Practical heuristic. If a model explains everything equally well, it usually explains nothing. Useful models make sharp commitments about mechanisms, variables, and expected failures.

1.2 A common abstraction stack

Many courses on higher brain function move across a recurring abstraction ladder: behavior and task structure, latent cognitive variables, circuit or algorithmic mechanisms, and normative or optimization principles.

That ladder connects classical cognitive models to modern neural-network and dynamical-systems viewpoints. Reinforcement learning, attractor dynamics, and recurrent computation all make the same point in different mathematical languages: representation, dynamics, and objective functions have to be studied together.

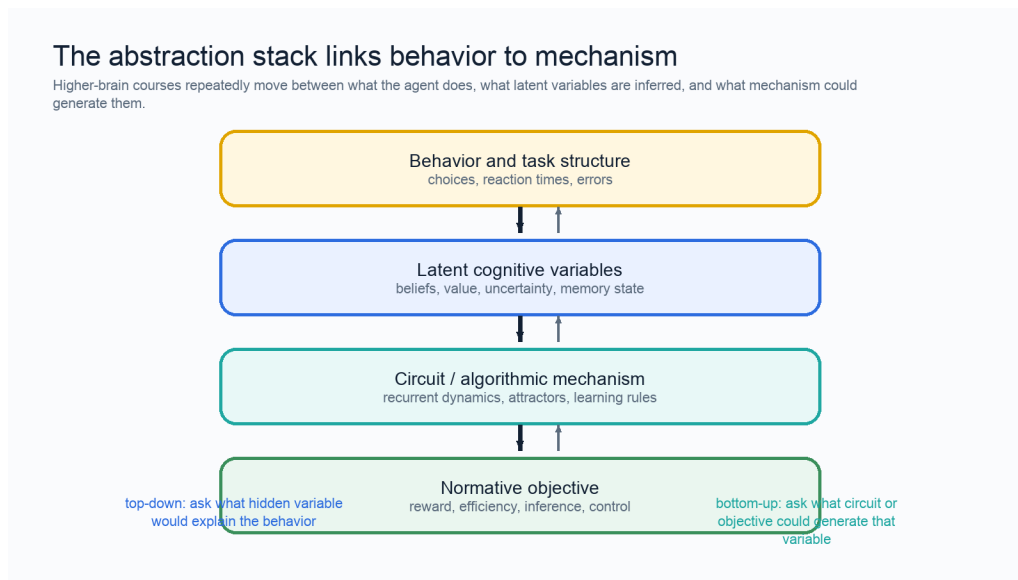


Figure 2: The abstraction ladder used throughout the course. Explanations move downward from observed behavior to hidden variables and mechanism, then upward again to ask whether the mechanism reproduces the behavior.

Generic state-space view.

$$x_{t+1} = f(x_t, u_t; \theta)$$

This form is broad enough to include recurrent neural networks, drift-diffusion style updates, working-memory attractors, and many cognitive-state models.

1.3 Reusable model code

Executable examples should make modeling assumptions visible. Useful code modules include simulation utilities for canonical cognitive tasks, fitting routines for simple latent-state models, plotting helpers for trajectories and phase portraits, and small reading replications that test whether a model reproduces the qualitative phenomenon it claims to explain.

1.4 Next steps

Each conceptual block should end with a compact model checklist: the state variables, the objective or update rule, the measurable predictions, and the most likely confound. This makes later

State-space models make hidden cognition explicit

The latent state summarizes what the system currently knows, while inputs push that state forward in time.

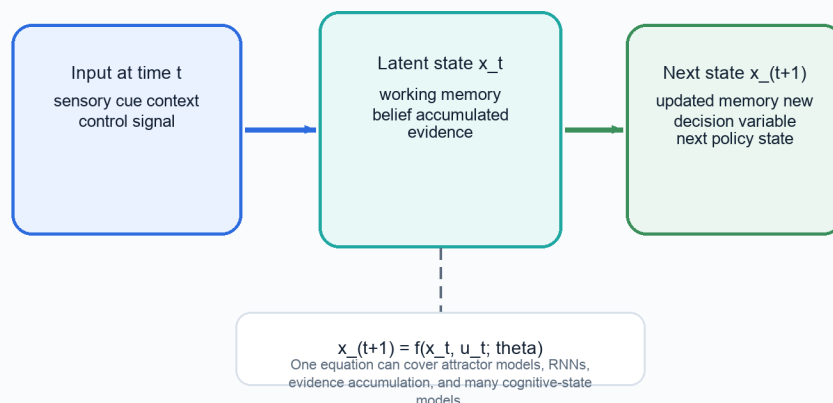


Figure 3: State-space intuition. The hidden state carries forward what the system currently “knows,” inputs perturb that state, and the update rule determines the next internal configuration.

comparisons between attention, memory, decision, and control models precise rather than purely verbal.

2 Lecture 1: Visual Perception and Deep Learning

These notes cover the opening perception lecture of *Models of Higher Brain Function*. The lecture uses vision as the first concrete example of a higher-brain computation: the sensory input is high-dimensional, noisy, and ambiguous, but behavior requires a stable internal description of objects and scenes. Deep neural networks enter the lecture not as a fashionable add-on, but as a way to make the representation problem explicit. They give us a concrete family of models that can solve large-scale object recognition tasks and can then be compared with the ventral visual stream.

2.1 Why vision belongs in a course on higher brain function

The lecture opens with a basic question: what does a visual system need to know about the world? At the level of behavior, the answers are simple enough:

- what is out there,
- where it is,
- and what it is likely to do next.

Those questions already show why vision is not just a matter of registering light intensity. A retinal image is only a pattern of local measurements. Perception requires the system to organize that pattern into useful latent variables such as object identity, position, motion, and behavioral relevance.

2.1.1 Why this problem is harder than it feels

One theme from the lecture is Moravec’s paradox: tasks that feel effortless to humans often turn out to be extremely demanding computationally. Recognizing a cup after it has moved, rotated, changed scale, or appeared under different illumination feels trivial in everyday life, but it is difficult because the raw sensory input changes enormously while the object identity stays the same.

This is why visual perception is treated here as a problem of inference rather than mere detection. The brain must move from a changing image to a useful internal representation of the world.

2.2 Invariant object recognition

The core challenge identified in the lecture is *invariant object recognition*. A useful visual system should keep track of what an object is even when many superficial aspects of the image change.

2.2.1 Why pixels are a bad format for recognition

The lecture follows the argument from DiCarlo and Cox that the raw image is a poor coordinate system for object recognition. Pixel values are highly sensitive to:

- translation across the retina,
- viewing angle,
- distance and scale,
- illumination,
- occlusion,
- and background clutter.

So two pictures of the same object can be very far apart in pixel space, while pictures of different objects may overlap heavily in their low-level image statistics. The slides describe this by saying that object manifolds are “hopelessly tangled up” in the original sensory coordinates.

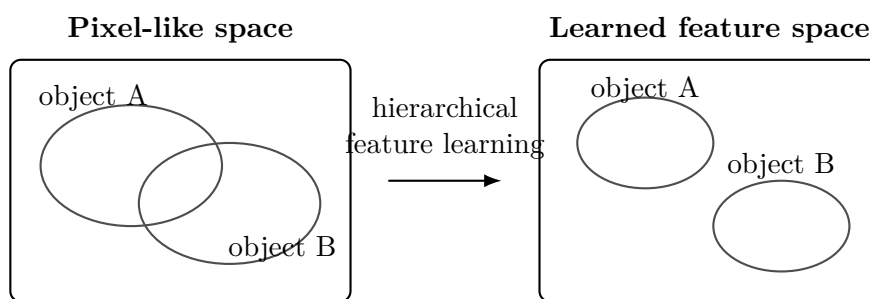


Figure 4: The geometric picture emphasized in the lecture. In raw image coordinates, object manifolds are tangled together. A good representation transforms them into a space where object identity is easier to separate.

2.2.2 What makes a representation useful

The lecture’s answer is that a “good” neural representation should vary gradually with nuisance variables such as position, illumination, or angle, while making object identity easier to read out. In

other words, recognition becomes easier when the representation performs a kind of *disentangling*: factors that should be treated as irrelevant for identification have a smaller effect on the code than they do in the original image.

This idea is important because it shifts the focus away from raw accuracy and toward representational geometry. The real question is not only whether the system labels an image correctly, but whether it transforms sensory input into coordinates that make the task easy for later stages.

2.3 The ImageNet turning point

The lecture then turns from the abstract recognition problem to the modern deep-learning solution. The key historical moment is the ImageNet era. ImageNet supplied a large-scale benchmark with roughly fifteen million labeled images spanning one thousand categories. This mattered because it forced models to solve recognition at a realistic visual scale rather than on small toy datasets.

2.3.1 Why AlexNet changed the conversation

AlexNet’s 2012 ImageNet result is presented as the turning point. Its top-5 error was 16.4%, while the next-best system remained at 26.2%. The significance of that result was not just the leaderboard gap. It showed that a deep, trainable convolutional network could learn useful visual features directly from data instead of depending mainly on hand-engineered descriptors.

The lecture also describes the “magic sauce” behind this success. Several ingredients had to come together at once:

- large labeled datasets,
- data augmentation,
- substantial computing power,
- convolution and weight sharing,
- rectified linear units,
- pooling and downsampling,
- and architectures deep enough to build hierarchical features.

So deep learning did not succeed because of one isolated trick. It succeeded because data, optimization, hardware, and architecture started to reinforce one another.

2.3.2 From AlexNet to ResNet

The lecture places AlexNet in a broader architectural sequence. VGG-style networks increased depth and regularized the use of small convolutions. ResNets then made very deep models trainable by adding skip connections. Instead of asking each block to learn an entirely new transformation, a residual block can learn a correction to the identity map. This helps with vanishing gradients and with the degradation in performance that appears when very deep plain networks become hard to optimize.

The slides make this concrete with the ImageNet benchmark again: ResNets reached depths above one hundred layers and pushed error rates below typical human performance on that dataset. The

lecture's point is not that ImageNet is a perfect model of human vision, but that these models had become competent enough that their internal representations became scientifically interesting.

2.4 What deep networks compute

At the mathematical level, the lecture describes a deep network as a concatenation of linear-nonlinear transformations. A single layer takes an input vector $\vec{x}$, applies an affine transformation, and then a nonlinearity:

$$\vec{y} = \phi(W\vec{x} + \vec{b}).$$

Here W is the weight matrix, $\vec{b}$ is a bias term, and ϕ is a nonlinear activation function.

2.4.1 Why nonlinearity matters

If every layer were purely linear, then stacking many layers would still amount to one overall linear transformation. Depth would not buy us much representational power. Nonlinearity is what lets the network bend and reshape the input space so that later layers can express more abstract and more separable features.

The lecture's qualitative story is straightforward:

- early layers detect simple local patterns such as oriented edges or contrast changes,
- intermediate layers combine those into textures or parts,
- later layers become more useful for category-level distinctions.

This staged abstraction is exactly why deep networks became plausible models of a hierarchical sensory pathway.

2.4.2 Why convolution is useful for images

Convolutional layers add two inductive biases that fit images well.

Local receptive fields. Each filter only sees a small image patch at a time. This reflects the fact that nearby pixels are strongly related and that many informative visual structures are local.

Weight sharing. The same filter is applied at many spatial positions. This makes it possible to detect the same local pattern wherever it appears and reduces the total number of parameters that must be learned.

Pooling and striding then expand the effective receptive field and make the representation less sensitive to small translations. This does not create perfect invariance, but it pushes the representation in the right direction for object recognition.

2.4.3 Backpropagation and the credit-assignment problem

Training is described in the lecture through backpropagation and stochastic gradient descent. A loss function measures the discrepancy between the network output and the target label, and gradients are propagated backward through the chain of nonlinear functions. A typical parameter update has the form

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w},$$

where $\mathcal{L}$ is the loss and η is the learning rate.

This solves the engineering problem of *credit assignment*: it tells the system how each parameter contributed to the final error. The lecture is careful, however, not to overinterpret this as a biological mechanism. Exact backpropagation, large labeled datasets, and perfectly shared convolutional filters are all biologically questionable. So deep networks are useful models of visual computation without being literal copies of cortical learning.

2.5 Deep networks as models of ventral visual cortex

Once deep networks became good at object recognition, the next question was whether their internal representations looked at all like those found in ventral visual cortex. The lecture emphasizes that the comparison should not be made unit by unit. A hidden unit in a trained network is not expected to correspond neatly to one biological neuron, and different random initializations of the same architecture need not preserve any unique one-to-one matching.

2.5.1 Why compare representational geometry

The more useful comparison is at the level of *representational similarity*. For a collection of stimuli, one can compute a response pattern for each stimulus and then ask how dissimilar those patterns are from one another. This yields a representational dissimilarity matrix:

$$\text{RDM}_{ij} = 1 - \text{corr}(r_i, r_j).$$

Here r_i and r_j denote the response vectors evoked by stimuli i and j .

The important point is that two systems can have similar representational geometry even if their individual units are not aligned. This is why representational similarity analysis became so useful in comparing human IT, monkey IT, and deep-network hidden layers.

2.5.2 The RSA construction

Suppose the experiment presents n stimuli. For one system, such as a cortical area or a neural-network layer, collect the responses into a matrix

$$R = \begin{bmatrix} r_1^\top \\ r_2^\top \\ \vdots \\ r_n^\top \end{bmatrix} \in \mathbb{R}^{n \times p}.$$

The row $r_i \in \mathbb{R}^p$ is the population response to stimulus i . In a brain dataset, the p dimensions might be voxels, recording sites, or neurons. In a network, they might be hidden units or feature channels.

RSA then replaces the response matrix by an $n \times n$ matrix of pairwise dissimilarities:

$$D_{ij} = d(r_i, r_j).$$

The function d defines what kind of representational difference is being measured. The most common choice in this lecture is correlation distance,

$$d_{\text{corr}}(r_i, r_j) = 1 - \frac{(r_i - \bar{r}_i \mathbf{1})^\top (r_j - \bar{r}_j \mathbf{1})}{\|r_i - \bar{r}_i \mathbf{1}\|_2 \|r_j - \bar{r}_j \mathbf{1}\|_2}.$$

This quantity ignores the overall response scale and asks whether the two response *patterns* have the same shape across units. A small value means that the two stimuli evoke similar population patterns; a large value means that they are represented far apart.

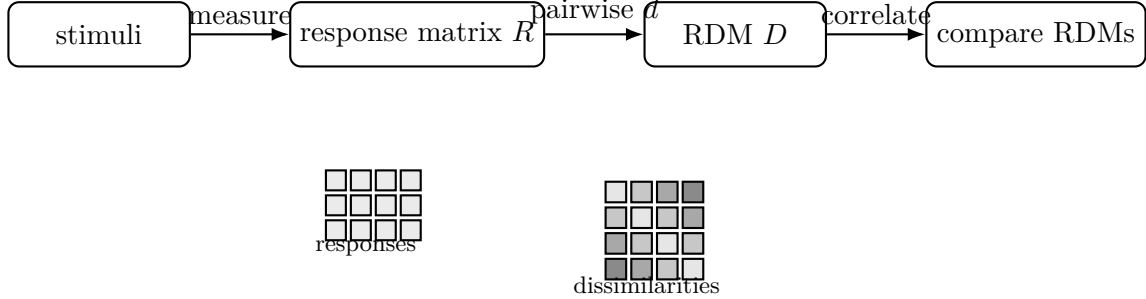


Figure 5: Representational similarity analysis turns high-dimensional response patterns into a matrix of stimulus-by-stimulus distances. The comparison between brain and model happens at the level of these distance matrices.

Other distance choices answer slightly different questions:

- Euclidean distance, $d_{\text{Euc}}(r_i, r_j) = \|r_i - r_j\|_2$, keeps absolute response magnitude and is therefore sensitive to scaling.
- Mahalanobis distance, $d_{\text{Mah}}(r_i, r_j) = (r_i - r_j)^\top \Sigma^{-1} (r_i - r_j)$, downweights noisy or correlated response dimensions through the covariance matrix Σ .
- Rank-based distances ignore exact metric values and ask whether the ordering of near and far stimulus pairs is preserved.

So an RDM is not just a visualization. It is a precise mathematical summary of the geometry induced by a representation on a fixed stimulus set.

2.5.3 Comparing a brain RDM with a model RDM

The second step is to compare two RDMs. Let D^{brain} be the RDM from a cortical area and D^{model} be the RDM from a network layer. Because both matrices are symmetric and have zeros on the diagonal, one usually vectorizes only the upper triangle:

$$v(D) = (D_{12}, D_{13}, \dots, D_{1n}, D_{23}, \dots, D_{n-1,n}).$$

The RSA score can then be written as

$$\rho_{\text{RSA}} = \text{corr}(v(D^{\text{brain}}), v(D^{\text{model}})).$$

In practice this is often a Spearman rank correlation, because researchers may care more about whether the model preserves the ordering of representational distances than about matching every numerical distance exactly.

This is the main conceptual advantage. The number of units in the two systems can be different, and their axes need not correspond. A cortical area might be measured with $p = 500$ voxels and a network layer might have $q = 4096$ features. RSA can still compare them because both are reduced to the same $n \times n$ stimulus geometry.

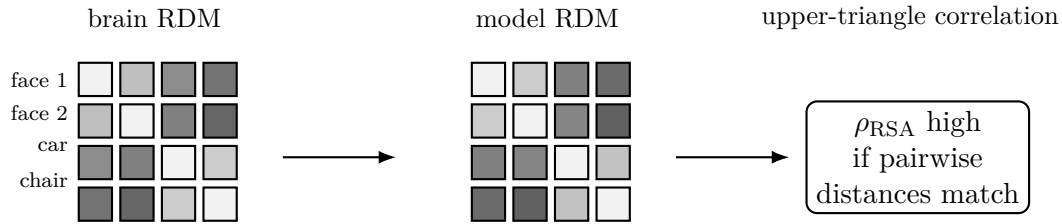


Figure 6: RSA is a second-order comparison. It does not ask whether one model unit equals one neuron. It asks whether pairs of stimuli that are close or far apart in cortex are also close or far apart in the model.

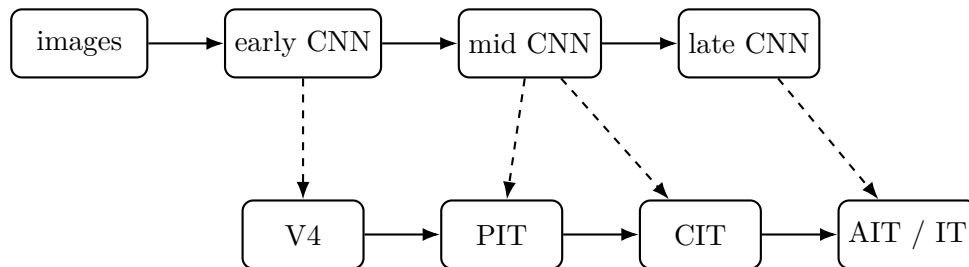


Figure 7: The comparison strategy highlighted in the lecture. Rather than matching units one by one, researchers compare which network layers best predict or resemble activity in successive ventral-stream areas.

2.5.4 How to interpret an RSA result

An RSA match should be read as evidence about representational geometry, not as a claim that the mechanisms are identical. A high RSA score says that the two systems place the tested stimuli in a similar pattern of relative distances. It does not by itself say that the same features were learned, that the same learning rule was used, or that the same computations occur internally.

There are two practical cautions:

- The stimulus set matters. An RDM built from faces, animals, tools, and places tests a different geometry than an RDM built only from small pose changes of one object.
- The noise ceiling matters. If repeated brain measurements are themselves only moderately correlated, then no model can be expected to reach a perfect RSA score.

This is why the lecture treats RSA as a powerful but limited bridge between deep learning and neuroscience. It gives a mathematically clean way to ask whether a model layer has the same *shape of representation* as a cortical area, while still leaving open the mechanistic question of how that representation is produced.

2.5.5 What the comparisons show

The lecture highlights a consistent empirical pattern: supervised deep models tend to explain the geometry of IT responses better than unsupervised alternatives, and models that are better optimized for object recognition often provide better predictions of neural data. This does not mean that cortex literally implements AlexNet or ResNet. It means that solving the right task appears to push artificial systems toward internal representations that partially resemble those found in the

brain.

That is a strong scientific claim. It suggests that task demands and representational constraints are deeply connected: if a system must solve invariant object recognition well, some aspects of its internal geometry may be forced into a similar shape, whether the system is biological or artificial.

2.6 Deep networks as experimental tools

The lecture goes further than model comparison and treats deep networks as practical tools for neuroscience experiments. The Bashivan-style examples make this especially clear. The logic is:

1. train a convolutional network on a large visual task,
2. learn a mapping from model features to recorded neural responses,
3. optimize the input image to maximize the predicted response of a chosen neuron or population,
4. and then test the synthesized image experimentally.

This matters because a differentiable model can be used not only to *explain* old data but also to *design* new stimuli. If the model captures something important about a neuron's preferred feature geometry, then the optimized image should strongly drive that neuron. The lecture discusses V4 experiments in which CNN-optimized images indeed strongly activated recorded neural sites.

So the deep model becomes more than a classifier. It becomes a hypothesis generator for what features a neural population is actually sensitive to.

2.7 What deep networks still miss

The lecture closes with several cautions. Modern deep networks are highly effective at image classification, but that does not make them complete theories of biological vision.

2.7.1 Adversarial fragility

Deep networks can be fooled by adversarial perturbations: small image changes that are nearly imperceptible to humans but cause a dramatic change in the predicted label. This shows that strong benchmark performance does not automatically imply human-like robustness or understanding.

2.7.2 Biological caveats

Several ingredients of standard deep learning remain biologically questionable:

- exact backpropagation,
- extensive supervised labels,
- strict convolutional weight sharing,
- and mostly feedforward inference in systems whose biological counterparts are strongly recurrent.

These caveats do not make deep networks unhelpful. They simply define the proper level of interpretation. Deep networks are useful computational models of visual recognition, and they may capture important aspects of cortical representation, but they are still abstractions.

2.8 Summary of concepts introduced in Lecture 1

The lecture introduced several ideas that should be kept distinct:

Concept	Role in the lecture
Invariant object recognition	keep identity stable across natural image changes
Disentangling	transform tangled image manifolds into easier features
ImageNet	large-scale benchmark that made modern comparison possible
Convolution	local receptive fields plus weight sharing for images
Backpropagation	practical credit-assignment method for deep networks
RSA / RDMs	compare vectorized stimulus-distance matrices across brains and models
CNN-optimized stimuli	use models to generate experimental probes
Adversarial examples	reveal limits of current deep-network robustness

2.9 Conceptual takeaways from Lecture 1

The opening vision lecture establishes the framework used later in the course.

- The main difficulty of vision is not detecting light, but constructing a representation that supports stable object recognition.
- Raw images are a bad format for recognition because nuisance variables strongly distort pixel values.
- Deep networks matter because they provide a concrete way to learn hierarchical feature spaces that partially solve this representation problem.
- The success of deep learning depends on a combination of architecture, data, optimization, and compute rather than on one isolated trick.
- Model evaluation can proceed on several levels at once: recognition performance, similarity to cortical representations, and usefulness for generating new experiments.
- High benchmark accuracy is not the same as biological plausibility or human-like robustness.

These points motivate the broader theme of the course: higher brain function can often be studied by asking what representation a system needs in order to solve a demanding cognitive task.

Sources mentioned in Lecture 1

- J. J. DiCarlo and D. D. Cox (2007), “Untangling invariant object recognition,” *Trends in Cognitive Sciences*.
- A. Krizhevsky, I. Sutskever, and G. E. Hinton (2012), “ImageNet classification with deep convolutional neural networks,” *NeurIPS*.
- K. He, X. Zhang, S. Ren, and J. Sun (2016), “Deep residual learning for image recognition,” *CVPR*.
- N. Kriegeskorte, M. Mur, and P. Bandettini (2008), “Representational similarity analysis,” *Frontiers in Systems Neuroscience*.
- S.-M. Khaligh-Razavi and N. Kriegeskorte (2014), “Deep supervised, but not unsupervised, models may explain IT cortical representation,” *PLoS Computational Biology*.

- D. L. K. Yamins and J. J. DiCarlo (2016), “Using goal-driven deep learning models to understand sensory cortex,” *Nature Neuroscience*.
- P. Bashivan, K. Kar, and J. J. DiCarlo (2019), “Neural population control via deep image synthesis,” *Science*.
- I. J. Goodfellow, J. Shlens, and C. Szegedy (2015), “Explaining and harnessing adversarial examples,” *ICLR*.

3 Lecture 2: Models of Attention

Attention is the first lecture topic in which the course moves from representation to selection. Vision already gave us the problem of constructing useful internal variables from high-dimensional input. Attention asks a different question: when many representations are simultaneously possible, which ones should receive processing priority, stronger gain, memory access, or control over action?

The lecture treats attention as a family of mechanisms rather than as a single faculty. There is overt attention, where the eyes or sensors are oriented toward an object, and covert attention, where a location or feature is prioritized without moving the eyes. There is exogenous attention, which is captured by the stimulus, and endogenous attention, which is guided by task goals. There is also spatial, feature-based, object-based, local, and global attention. The common computational problem is selection under limited processing capacity.

Two model families organize the lecture:

- saliency-map models, which explain early bottom-up attentional capture and the first few fixation points;
- biased-competition models, which explain how attention changes neural population responses when multiple stimuli compete.

3.1 The behavioral anchor: pop-out and serial search

The simplest behavioral distinction is between pop-out and serial visual search. Let n denote the number of items in a display, and let $T(n)$ be the reaction time required to find the target.

In a pop-out display, the target differs from distractors by a simple low-level feature such as color or orientation. The search-time slope is close to zero:

$$T_{\text{pop}}(n) \approx T_0 + \epsilon n, \quad \epsilon \approx 0.$$

The target is found almost as quickly among many distractors as among few distractors.

In serial search, the target is defined by a conjunction or by a relation that does not create a unique low-level feature peak. Search time increases with display size:

$$T_{\text{serial}}(n) \approx T_0 + \alpha n, \quad \alpha > 0.$$

If search is self-terminating and the target is present, the expected number of inspected items is roughly $n/2$. If the target is absent, all n items must be checked:

$$T_{\text{present}}(n) \approx T_0 + \alpha \frac{n}{2}, \quad T_{\text{absent}}(n) \approx T_0 + \alpha n.$$

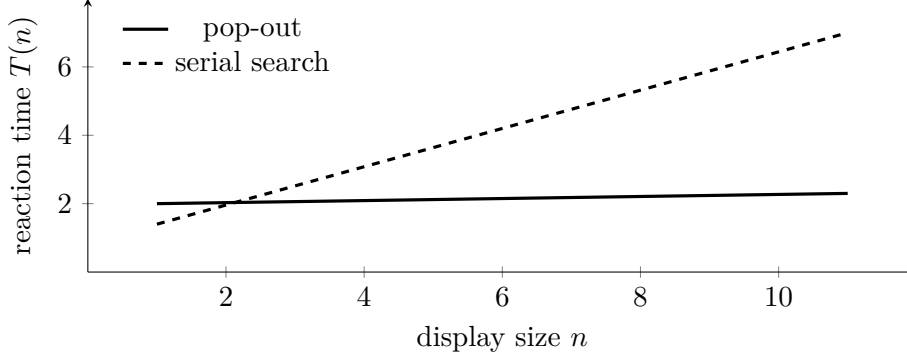


Figure 8: The behavioral signature of attention in search tasks. Pop-out has an almost flat reaction-time slope because one item creates a distinctive priority peak. Serial search has a positive slope because many items must be inspected in sequence.

This distinction is the bridge to saliency maps. Pop-out occurs when the visual field contains one strong priority peak. Serial search occurs when many items have similar priority or when the target is defined by a rule that the low-level image alone does not isolate.

3.2 The spotlight as a minimal gain model

The spotlight metaphor says that attention selects a restricted spatial region. A compact mathematical version is a spatial gain field $A(x)$ applied to a visual representation $R(x)$:

$$R_{\text{att}}(x) = A(x)R(x).$$

If the attended location is x_0 , a narrow focus can be written as

$$A(x) = 1 + g \exp\left(-\frac{\|x - x_0\|^2}{2\sigma_A^2}\right).$$

Here g is the strength of attentional gain, and σ_A is the size of the attended region.

This captures the basic idea of covert spatial attention: a location can receive more effective processing even if the eyes remain fixed elsewhere. But it is incomplete as a general theory. Attention can select features, objects, task rules, or global structure, not only regions of visual space. The spotlight is therefore useful as a first gain model, not as a complete account.

3.3 Saliency maps

Saliency-map models ask which image location is locally distinctive. The key point is that saliency is not raw feature value; it is feature contrast. A red object is not automatically salient. It becomes salient when it is red among non-red objects.

Let $I(x, y)$ denote the image. A feature channel k , such as intensity, color opponency, or orientation, is extracted by a multiscale filter:

$$F_k(\cdot, \cdot, s) = h_{k,s} * I,$$

where s is scale, $h_{k,s}$ is a feature-selective filter, and $*$ denotes convolution.

3.3.1 Center-surround contrast

For a center scale c and a larger surround scale s , a feature-contrast map can be written as

$$D_{k,c,s}(x, y) = |F_k(x, y, c) - \text{resize}(F_k(\cdot, \cdot, s))(x, y)|.$$

This is the multiscale version of a center-surround operation. It compares a local measurement with the surrounding context. A continuous approximation is a difference-of-Gaussians contrast:

$$D_k(x, y) \approx |(G_{\sigma_c} * F_k)(x, y) - (G_{\sigma_s} * F_k)(x, y)|, \quad \sigma_s > \sigma_c.$$

The intuition is direct. A single vertical bar among horizontal bars creates large orientation contrast. A uniformly vertical field has high vertical-feature activity but low contrast, so it does not pop out.

3.3.2 Normalization

The Itti-Koch model normalizes feature maps so that maps with one strong winner are emphasized over maps with many equivalent peaks. If M is a contrast map, first scale it to $[0, 1]$:

$$\widetilde{M} = \frac{M - \min M}{\max M - \min M}.$$

Let $m_{\max}$ be the global maximum of $\widetilde{M}$, and let $\bar{m}_{\text{local}}$ be the average value of the other local maxima. A simple version of the normalization is

$$\mathcal{N}(M) = \widetilde{M} (m_{\max} - \bar{m}_{\text{local}})^2.$$

The multiplicative factor is large when one peak dominates and small when many peaks are similar. This is why pop-out displays produce a strong saliency signal while serial-search displays often do not.

3.3.3 From feature maps to a master map

Feature maps are combined into conspicuity maps, and conspicuity maps are combined into a single master saliency map. For channel k ,

$$C_k(x, y) = \sum_{c,s} \mathcal{N}(D_{k,c,s})(x, y).$$

The final saliency map is a weighted sum:

$$S(x, y) = \sum_k w_k \mathcal{N}(C_k)(x, y).$$

In a purely bottom-up model, the weights w_k are fixed or uniform. Limited top-down influence can be added by increasing w_k for task-relevant feature channels. For example, a search for a green oblique bar can increase the weights for green and oblique features. This helps, but it is still weak compared with the semantic and goal-directed structure of real behavior.

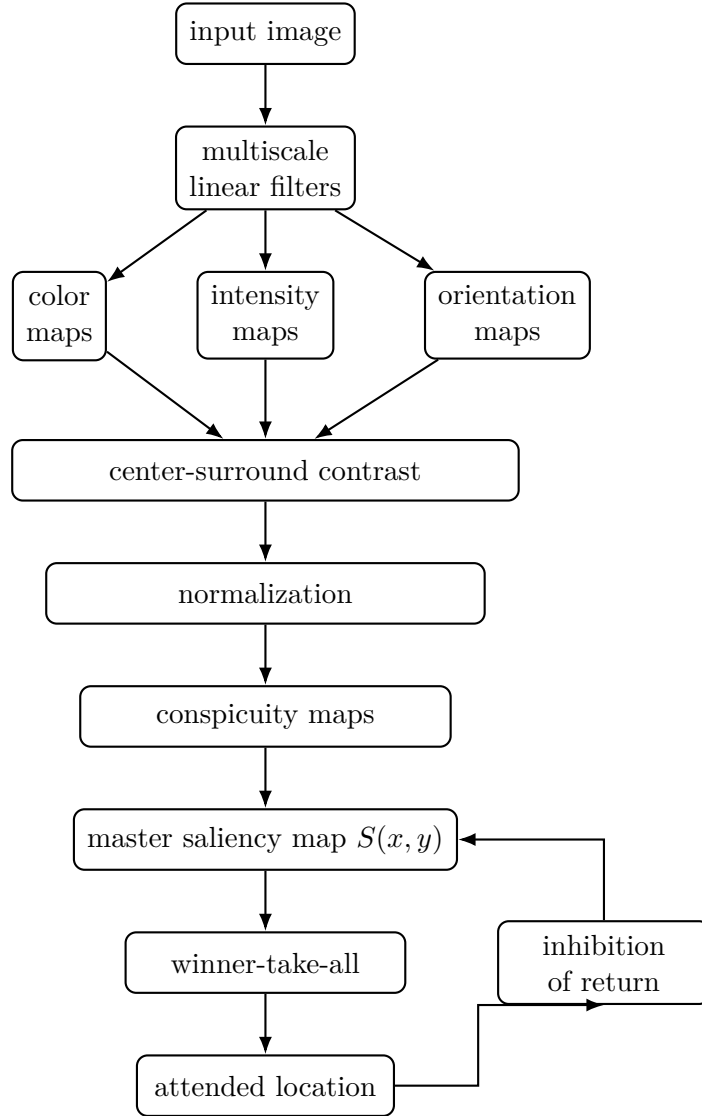


Figure 9: A saliency-map model. The image is decomposed into low-level feature maps, transformed into center-surround contrast maps, normalized, and pooled into a master priority map. Winner-take-all selects the next location, while inhibition of return suppresses recently selected locations.

3.3.4 Winner-take-all and inhibition of return

The model selects the current priority peak:

$$\ell_t = \arg \max_{(x,y)} S_t(x, y).$$

After a location has been selected, inhibition of return reduces its local priority:

$$S_{t+1}(x, y) = S_t(x, y) - \gamma \exp\left(-\frac{\|(x, y) - \ell_t\|^2}{2\sigma_{\text{IOR}}^2}\right), \quad \gamma > 0.$$

The next fixation or attentional shift is then selected from the updated map. This mechanism turns a static priority map into a sequence of visited locations.

3.3.5 What saliency maps explain

Saliency-map models explain early attentional capture by low-level features and qualitatively reproduce the pop-out versus serial-search distinction. They can also account for the first few fixation points in images with little task structure.

Their limitation is equally important. Later viewing behavior depends on personal relevance, task demands, object meaning, memory, and reward. The lecture's comparison between bottom-up saliency models and learned fixation predictors makes this point: modern CNN-based predictors can absorb more top-down regularities from human saccade data, but then the model is no longer a purely bottom-up saliency theory.

3.4 Attention on the neural level

The second half of the lecture shifts from where attention goes to how attention changes neural responses. The empirical starting point is that neural responses are modulated by attention, and this modulation is stronger in higher visual areas such as V4 and IT than in early visual cortex.

A compact rate model writes the response of population i as

$$r_i = \phi(g_i s_i + b_i),$$

where s_i is the sensory drive, g_i is a multiplicative gain, b_i is an additive bias, and ϕ is a static nonlinearity. Attention can change gain, baseline, tuning width, or competition between populations. The biased-competition model focuses on the last of these.

3.5 Stimulus interactions and suppression

When two stimuli appear inside the same receptive field, the response to the pair is not generally the sum of the responses to each stimulus alone. The lecture summarizes this with a suppression index:

$$\text{SI} = 100 \frac{r_{\text{pair}} - r_{\text{best}}}{r_{\text{best}}}.$$

Here r_{best} is the response to the preferred stimulus alone, and r_{pair} is the response when the preferred and nonpreferred stimuli are shown together. If $\text{SI} < 0$, the distractor suppresses the response to the preferred stimulus. This suppression is the neural signature of competition.

3.6 Biased competition

The biased-competition model says that stimuli activate neural populations, those populations inhibit one another, and top-down attention biases the competition toward the task-relevant population. Attention does not create the representation from nothing. It changes which active representation wins.

Let $r_A(t)$ and $r_X(t)$ be firing rates for populations representing stimuli A and X . Let I_A and I_X be bottom-up sensory inputs, and let b_A and b_X be top-down attentional biases. A minimal two-population model is

$$\begin{aligned}\tau \frac{dr_A}{dt} &= -r_A + \phi(I_A + b_A + Er_A - Lr_X), \\ \tau \frac{dr_X}{dt} &= -r_X + \phi(I_X + b_X + Er_X - Lr_A).\end{aligned}$$

The parameter $E > 0$ captures recurrent self-excitation, $L > 0$ captures lateral inhibition, and τ is a time constant.

If attention is directed to A , then $b_A > b_X$. The top-down signal tilts the competition toward the population representing A . This is why the same physical display can produce different neural responses depending on the attended target.

3.6.1 Sum and difference variables

In the linear active regime, where $\phi(z) = z$, the equilibrium obeys

$$\begin{aligned}(1 - E)r_A + Lr_X &= I_A + b_A, \\ Lr_A + (1 - E)r_X &= I_X + b_X.\end{aligned}$$

Adding the equations gives

$$(1 - E + L)(r_A + r_X) = I_A + I_X + b_A + b_X.$$

If total sensory input and total attentional drive are held fixed, the sum of the rates is approximately constrained. This matches the lecture's emphasis on a roughly constant total activity in competitive dynamics.

Subtracting the equations gives

$$(1 - E - L)(r_A - r_X) = (I_A - I_X) + (b_A - b_X).$$

The difference $b_A - b_X$ directly changes the difference between the competing population activities. This is the mathematical core of biased competition: attention can redistribute activity rather than simply increasing all responses.

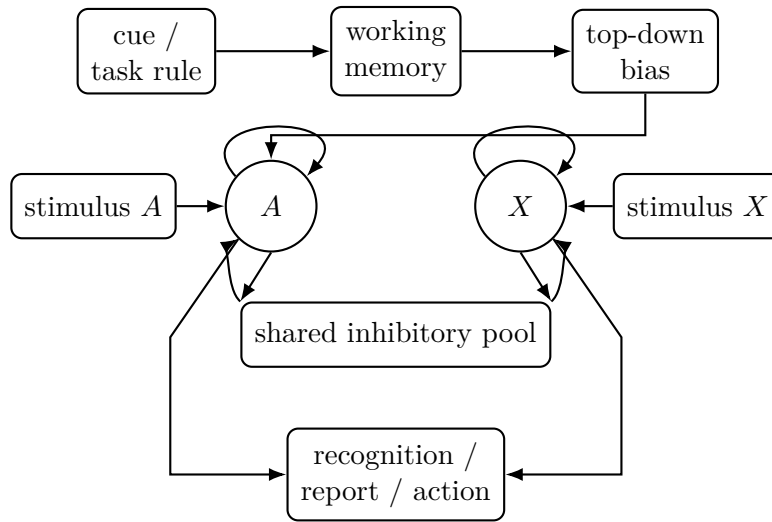


Figure 10: Biased competition. Sensory input activates several populations, recurrent excitation stabilizes each active representation, shared inhibition creates competition, and a top-down signal from the cue or task state biases one competitor.

3.6.2 Divisive normalization as a related form

A modern way to write competitive response modulation is divisive normalization:

$$r_i = R_{\max} \frac{(g_i I_i)^n}{\sigma^n + \sum_j (g_j I_j)^n}.$$

The numerator is the drive to population i . The denominator is a pooled activity term. Increasing the attentional gain g_i for one population increases its own response, but because all populations share the denominator, the change is relative to the rest of the population. This gives a clean mathematical bridge between attention, competition, and cortical normalization.

3.7 Cueing, delay activity, and target trials

The Chelazzi-style experiments in the lecture use a cue, a delay, and a later display containing target and distractor stimuli. The important observation is that neural responses depend on which stimulus is behaviorally relevant, even when the physical stimulus display is the same.

The model interpretation is:

- the cue establishes a working-memory state;
- this state supplies a top-down bias during the delay and stimulus presentation;
- the display activates multiple sensory populations;
- recurrent excitation and inhibition create competition;
- the bias favors the population representing the target.

So attention is not merely a late verbal label attached to a response. It is a dynamical variable that changes the evolution of neural activity during competition.

3.8 Relation to transformer self-attention

The lecture briefly connects biological attention to self-attention in machine learning. Transformer attention computes

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V.$$

For token i , this is

$$y_i = \sum_j \alpha_{ij} v_j, \quad \alpha_{ij} = \frac{\exp(q_i^\top k_j / \sqrt{d})}{\sum_m \exp(q_i^\top k_m / \sqrt{d})}.$$

The analogy is useful but limited. Both biological attention and transformer attention involve normalized, context-dependent weighting of information sources. In transformers, query-key similarity determines which values are routed into the next representation. In biological visual attention, task state and sensory competition determine which neural populations receive priority.

The mismatch is just as important. Transformer self-attention is not gaze, consciousness, or a literal cortical circuit. It usually has no explicit saccade policy, no inhibition of return, and no biological recurrent dynamics. The productive bridge is therefore not that transformers are brains, but that both systems use normalized selection to route limited representational resources.

3.9 Frontier directions

The classical saliency model uses hand-designed channels: color, intensity, and orientation. A modern learned priority model replaces these by features learned from fixation data, task behavior, or large-scale visual training. The computational question changes from asking which hand-designed feature contrast is locally distinctive to asking which learned visual and task variables predict where an agent will sample next.

Another frontier move is to replace winner-take-all selection with a probability distribution over fixations:

$$p(\ell_t = x) = \frac{\exp(S(x)/T)}{\sum_{x'} \exp(S(x')/T)}.$$

Low temperature T approximates hard winner-take-all. Higher temperature permits exploration. This connects attention to active vision and reinforcement learning, where a fixation is an action chosen for its expected information or reward.

At the circuit level, the frontier question is what the bias term b_i actually contains. It may combine prefrontal working-memory signals, parietal priority maps, thalamic routing, neuromodulatory gain, and task context:

$$b_i(t) = b_i^{\text{PFC}}(t) + b_i^{\text{parietal}}(t) + b_i^{\text{thalamic}}(t) + b_i^{\text{neuromod}}(t).$$

The simple biased-competition equation is therefore a compact abstraction of many mechanisms.

3.10 Summary of concepts introduced in Lecture 2

Concept	Role in the lecture
Pop-out	near-flat reaction-time slope caused by a unique low-level priority peak
Serial search	positive reaction-time slope when attention must inspect items sequentially
Spotlight model	spatial gain field over a visual representation
Saliency map	master priority map built from normalized feature contrasts
Center-surround contrast	local feature value compared with surrounding context
Inhibition of return	suppression of recently attended locations to support sequential exploration
Suppression index	measure of response reduction when competing stimuli share a receptive field
Biased competition	recurrent competition tilted by top-down task or memory signals
Divisive normalization	shared denominator that turns attentional gain into relative response modulation
Transformer attention	normalized routing analogy, not a literal biological mechanism

3.11 Conceptual takeaways from Lecture 2

- Attention is best treated as a family of selection mechanisms, not as one process.
- Saliency depends on contrast relative to context, not on raw feature magnitude.
- Pop-out and serial search differ behaviorally because their priority landscapes differ.

- Saliency-map models explain early bottom-up capture but are limited for task-rich and semantic behavior.
- Biased competition explains why the same stimulus can evoke different neural responses depending on the attended target.
- Top-down attention can redistribute activity under a competition or normalization constraint rather than simply increasing all firing rates.
- The bridge to modern machine learning is normalized, context-dependent routing; the gap is that biological attention is embodied, recurrent, and action-linked.

Sources mentioned in Lecture 2

- C. Koch and S. Ullman (1985), “Shifts in selective visual attention: towards the underlying neural circuitry,” *Human Neurobiology*.
- L. Itti, C. Koch, and E. Niebur (1998), “A model of saliency-based visual attention for rapid scene analysis,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- R. Desimone and J. Duncan (1995), “Neural mechanisms of selective visual attention,” *Annual Review of Neuroscience*.
- M. Usher and E. Niebur (1996), “Modeling the temporal dynamics of IT neurons in visual search,” *Neurocomputing*.
- L. Chelazzi and colleagues (1998), experiments on attention, memory, and competition in visual cortex.
- C. J. McAdams and J. H. R. Maunsell (1999), work on attentional modulation in visual cortex.
- A. Vaswani and colleagues (2017), “Attention is all you need,” *NeurIPS*.
- G. W. Lindsay (2020), review on attention in psychology, neuroscience, and machine learning.

4 Mathematical Synthesis: Higher Brain Function as Inference

Higher brain function is difficult because the variables of interest are latent. We observe images, actions, response times, choices, spike trains, and population activity; we infer beliefs, memories, task states, goals, and decisions. A good model makes that inference problem explicit.

4.1 Latent States

The basic abstraction is a state-space model:

$$z_{t+1} \sim p(z_{t+1} \mid z_t, a_t), \quad x_t \sim p(x_t \mid z_t), \quad a_t \sim \pi(a_t \mid z_t).$$

Here x_t is observed sensory or behavioral data, z_t is the latent cognitive state, and a_t is an action or report. The model says that cognition is not a direct label attached to the stimulus; it is an inferred state that evolves over time and helps predict both future observations and behavior.

Modeling intuition. The latent state should be just rich enough to make the future conditionally simpler. If knowing z_t makes past details irrelevant for predicting the next observation or action, then z_t has captured the right memory for the task.

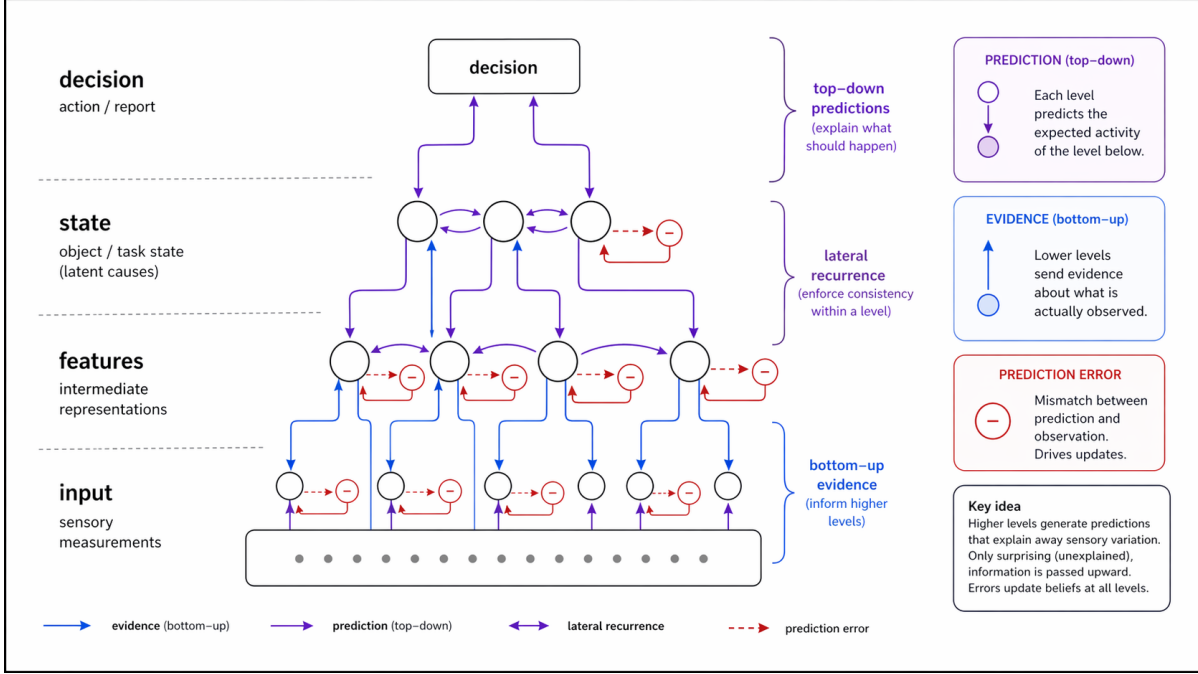


Figure 11: A compact view of hierarchical inference. Bottom-up evidence constrains higher latent states, top-down predictions explain expected sensory structure, lateral recurrence enforces consistency within a level, and prediction errors update the representation.

4.2 Prediction And Error

In a hierarchical model, higher levels send predictions downward and lower levels send mismatches upward. A minimal predictive-coding objective can be written as

$$E(z) = \frac{1}{2} \|x - g(z)\|_{\Sigma_x^{-1}}^2 + \frac{1}{2} \|z - \mu\|_{\Sigma_z^{-1}}^2.$$

The first term penalizes sensory prediction error. The second term penalizes deviation from prior expectations. Inference changes z to reduce the total error:

$$\dot{z} \propto -\nabla_z E(z).$$

This equation is not just an optimization rule. It gives a mechanistic story for recurrent cortical dynamics: activity settles toward a state that jointly respects sensory evidence and top-down expectations.

4.3 Representations As Coordinates

Perception becomes useful when the representation separates task-relevant variation from nuisance variation. If x is an image and $z = f_\theta(x)$ is a representation, then a downstream decision should be simpler in z -space than in pixel space:

$$y \approx h(z), \quad z = f_\theta(x).$$

Deep networks make this idea concrete. Early layers preserve local structure, intermediate layers combine features, and later layers tend to align more strongly with object or task variables. The scientific question is not only whether the model classifies correctly, but whether its internal geometry matches the invariances and error patterns of biological perception.

4.4 Decisions Couple Belief To Action

Higher-brain models often end with a decision rule. A common normative form is

$$a^* \in \arg \max_a \mathbb{E}[U(a, Z) \mid x_{1:t}].$$

The action is chosen under a posterior belief about the latent state Z . This is the bridge from perception to behavior: the system does not act on the raw stimulus, but on an uncertainty-weighted internal estimate of what the stimulus means for the task.